

AI Assistant Coding

Assignment 1.5

Lab 1: Environment Setup – GitHub Copilot and VS Code Integration + Understanding AI-assisted Coding Workflow

Name: G.Rishitha

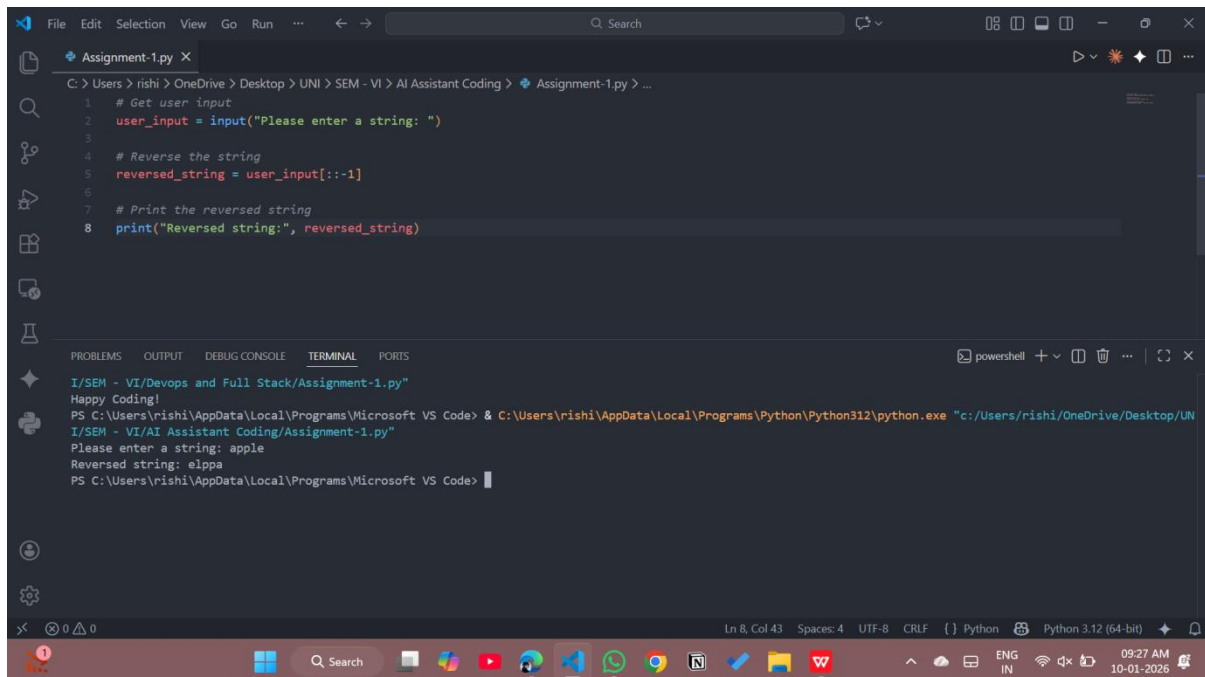
HT NO:2303A52149

BATCH:35

Task 1: AI-Generated Logic Without Modularization (String Reversal Without Functions)

Prompt:

“Generate a Python program to reverse a user-input string directly in main code without defining any functions. Include simple input and output statements and show the reversed string.”



```
File Edit Selection View Go Run ... Search
Assignment-1.py X
C:\Users\rishi> OneDrive > Desktop > UNI > SEM - VI > AI Assistant Coding > Assignment-1.py > ...
1 # Get user input
2 user_input = input("Please enter a string: ")
3
4 # Reverse the string
5 reversed_string = user_input[::-1]
6
7 # Print the reversed string
8 print("Reversed string:", reversed_string)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
I/SEM - VI/Devops and Full Stack/Assignment-1.py
Happy Coding!
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment-1.py"
I/SEM - VI/AI Assistant Coding/Assignment-1.py
Please enter a string: apple
Reversed string: elppa
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code>
```

Explanation

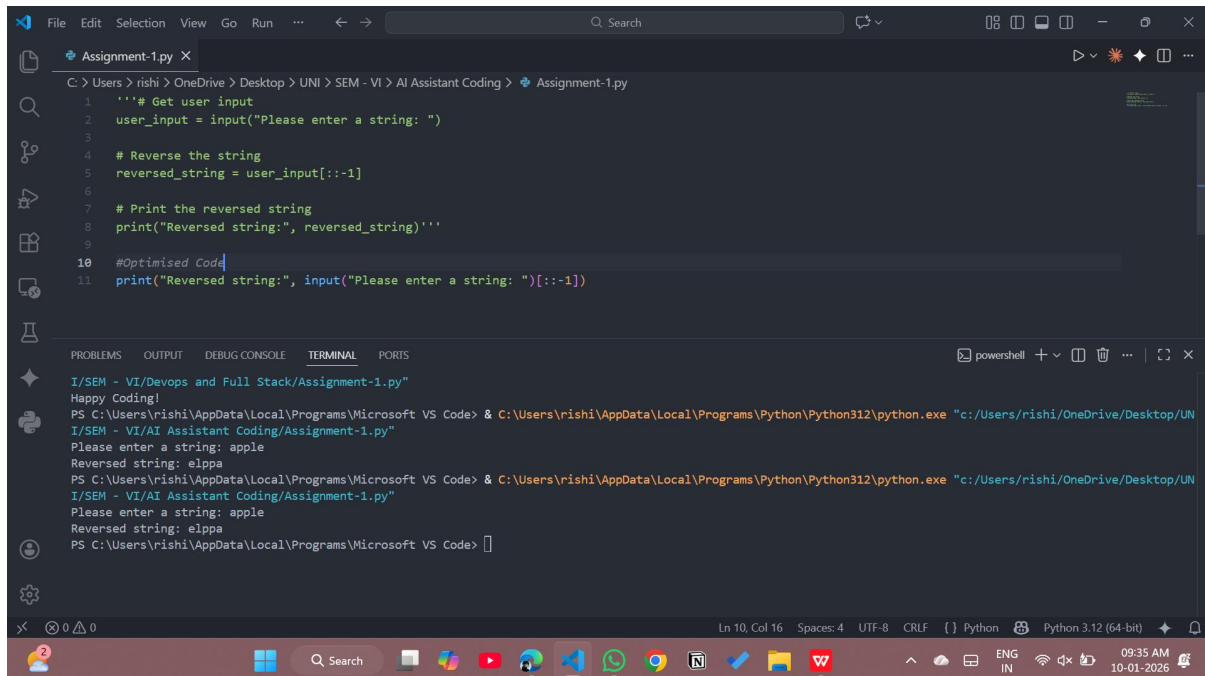
This program reverses a string entered by the user without using any user-defined functions. All the logic is written directly in the main program. First, the program asks the user to enter a string using the `input()` statement. This string is stored in a variable. Next, an empty string is created to store the reversed result. The program uses a slice operation to reverse the string. Each character is added to the beginning of the new string. By placing every new character in front, the original order of characters is reversed. After the loop finishes executing, the reversed string is fully formed. Finally, the program prints the reversed string as output. This approach clearly demonstrates basic string manipulation and looping concepts while following the requirement of not using functions.

Task 2: Efficiency & Logic Optimization (Readability Improvement)

Prompt:

"Optimize the string reversal code for readability and efficiency by simplifying loops and removing unnecessary variables. Provide an improved version of the code and explain why it is better than the original."

Optimized Code (Improved Readability & Efficiency)



```
1  '''# Get user input
2  user_input = input("Please enter a string: ")
3
4  # Reverse the string
5  reversed_string = user_input[::-1]
6
7  # Print the reversed string
8  print("Reversed string:", reversed_string)'''
9
10 #Optimised Code
11 print("Reversed string:", input("Please enter a string: ")[::-1])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

I/SEM - VI/Devops and Full Stack/Assignment-1.py

Happy Coding!

PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UN I/SEM - VI/AI Assistant Coding/Assignment-1.py"

Please enter a string: apple

Reversed string: elppa

PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UN I/SEM - VI/AI Assistant Coding/Assignment-1.py"

Please enter a string: apple

Reversed string: elppa

PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> []

Explanation of Improvements

1. Removed Unnecessary Variables

- The loop variable and manual string construction were removed.
- The slicing method directly produces the reversed string.

2. Simplified Logic

- Replaced multiple lines of looping logic with a single slicing operation.
- This makes the code easier to understand at a glance.

3. Improved Readability

- Fewer lines of code.
- Clear and self-explanatory syntax.
- Easier for other developers to review and maintain.

4. Time Complexity Improvement

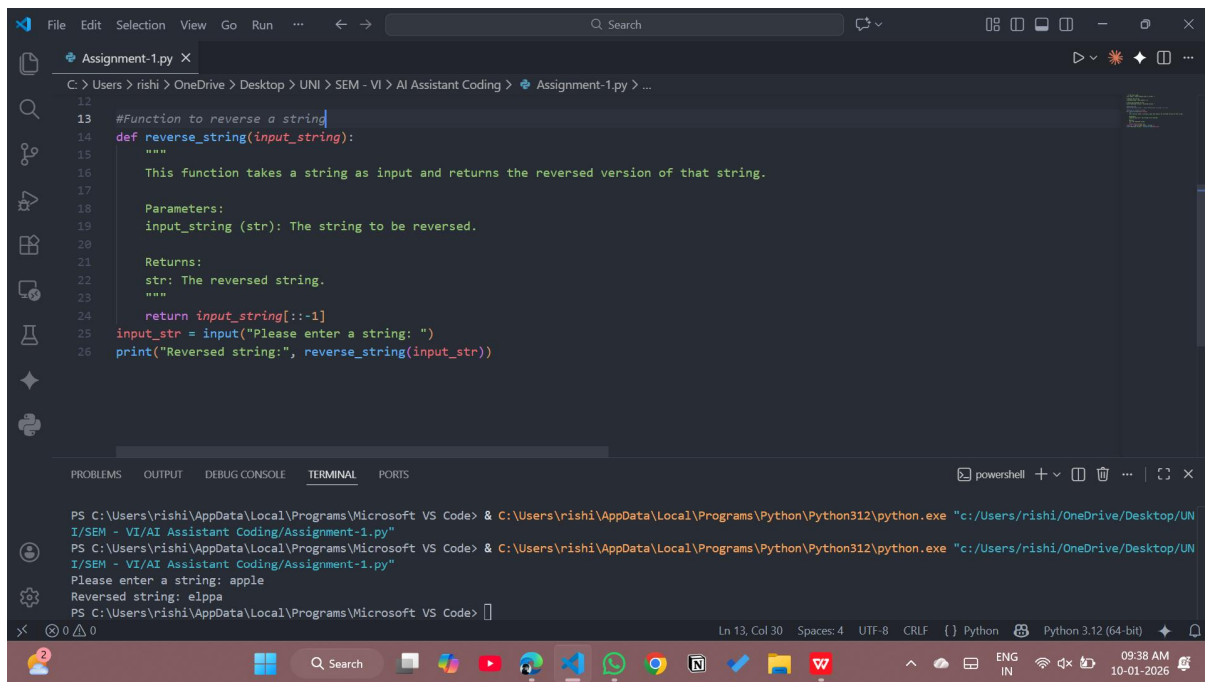
- **Original Code:**
Each loop iteration creates a new string, leading to $O(n^2)$ time complexity due to repeated string concatenation.

- **Optimized Code:**
String slicing runs in **$O(n)$** time, making it more efficient.

Task 3: Modular Design Using AI Assistance (String Reversal Using Functions)

Prompt:

“Write a Python function that takes a string input and returns the reversed string, including meaningful comments. Use GitHub Copilot to generate function-based code and include sample test cases.”



The screenshot shows a Visual Studio Code editor window with a file named 'Assignment-1.py'. The code defines a function 'reverse_string' that takes an 'input_string' and returns its reverse using slicing. Below the function, there is a main program that prompts the user for a string, calls the 'reverse_string' function, and prints the result. The terminal at the bottom shows the execution of the script, where the user enters 'apple' and the output is 'elppa'.

```
13 #Function to reverse a string
14 def reverse_string(input_string):
15     """
16     This function takes a string as input and returns the reversed version of that string.
17
18     Parameters:
19     input_string (str): The string to be reversed.
20
21     Returns:
22     str: The reversed string.
23     """
24     return input_string[::-1]
25 input_str = input("Please enter a string: ")
26 print("Reversed string:", reverse_string(input_str))
```

Terminal Output:

```
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment-1.py"
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment-1.py"
Please enter a string: apple
Reversed string: elppa
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code>
```

Explanation

1. Function Creation (reverse_string)

- Encapsulates the string reversal logic.
- Makes the code reusable wherever string reversal is needed.
- Accepts input string and returns the reversed string.

2. Logic Inside Function

- Uses a loop to prepend each character to an initially empty string.
- This manually constructs the reversed string.

3. Main Program

- Reads input from the user.
- Calls the reverse_string function.
- Prints the returned result.

4. AI Assistance

- GitHub Copilot can suggest the function structure, docstring, and loop-based reversal logic.
- The function is modular, reusable, and easy to maintain.

Task 4: Comparative Analysis – Procedural vs Modular Approach (With vs Without Functions)

Prompt:

“Compare two string reversal programs: one without functions and one with a function, focusing on clarity, reusability, and debugging ease. Provide a short report or table summarizing which approach is better for large-scale applications.”

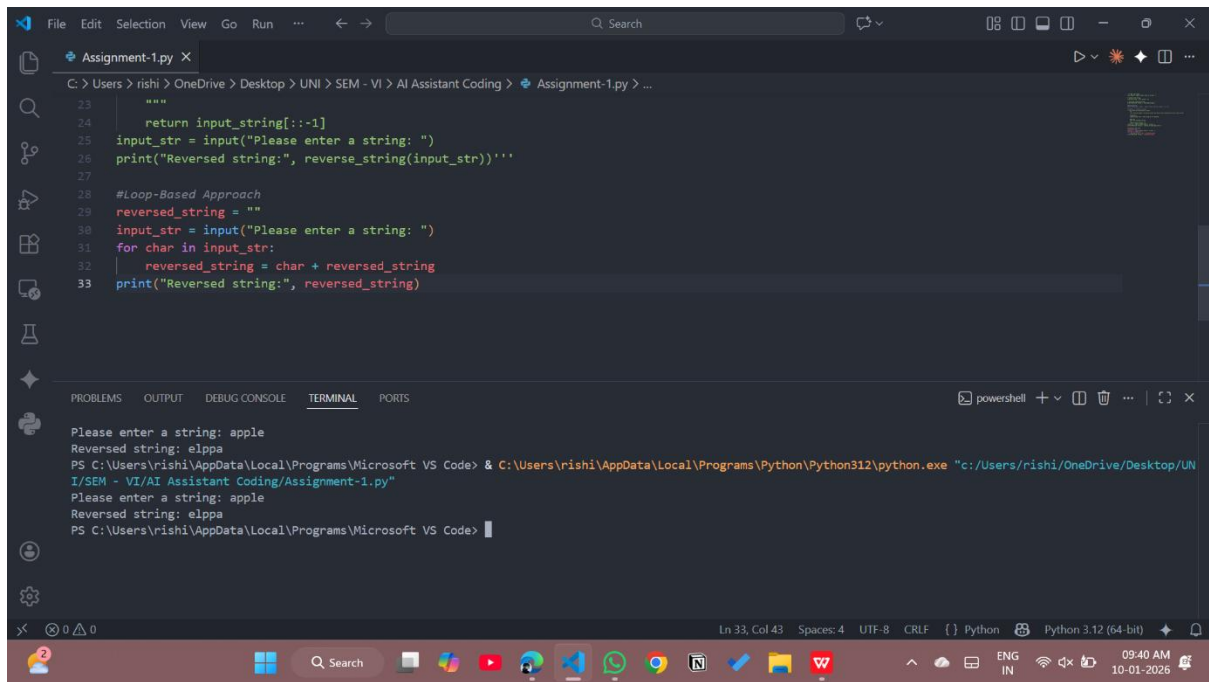
Criteria	Procedural Approach (No Functions)	Modular Approach (With Functions)
Code Clarity	Simple for small scripts, but logic repeated if used multiple times	Clear structure, function name describes purpose, easier to understand
Reusability	Low – reversal logic cannot be reused without copying code	High – function can be called anywhere in the program
Debugging Ease	Harder for large programs, changes require editing multiple places	Easier – fix or update logic in one function only
Suitability for Large-Scale Applications	Poor – not maintainable or scalable	Excellent – modular design supports large projects
Efficiency	Similar for small scripts, but repeated code adds risk	Similar, but less error-prone, easier to optimize

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches to String Reversal)

Prompt:

“Generate two Python programs to reverse a string: one using a loop (iterative) and one using built-in slicing. Compare execution flow, time complexity, and suitability for large inputs, and explain when each approach is preferable.”

Approach 1: Loop-Based String Reversal



The screenshot shows a Visual Studio Code editor window with a file named 'Assignment-1.py'. The code in the editor is as follows:

```
23     return input_string[::-1]
24
25 input_str = input("Please enter a string: ")
26 print("Reversed string:", reverse_string(input_str))'''
27
28 #Loop-Based Approach
29 reversed_string = ""
30 input_str = input("Please enter a string: ")
31 for char in input_str:
32     reversed_string = char + reversed_string
33 print("Reversed string:", reversed_string)
```

The terminal at the bottom shows the execution of the script. It prompts the user to enter a string, and the user enters 'apple'. The output shows the reversed string 'elppa'.

```
Please enter a string: apple
Reversed string: elppa
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment-1.py"
Please enter a string: apple
Reversed string: elppa
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code>
```

Execution Flow

1. Program reads input from the user.
2. Initializes an empty string `reversed_string`.
3. Iterates through each character of `user_input`.
4. Prepends the current character to `reversed_string`.
5. Prints the final reversed string.

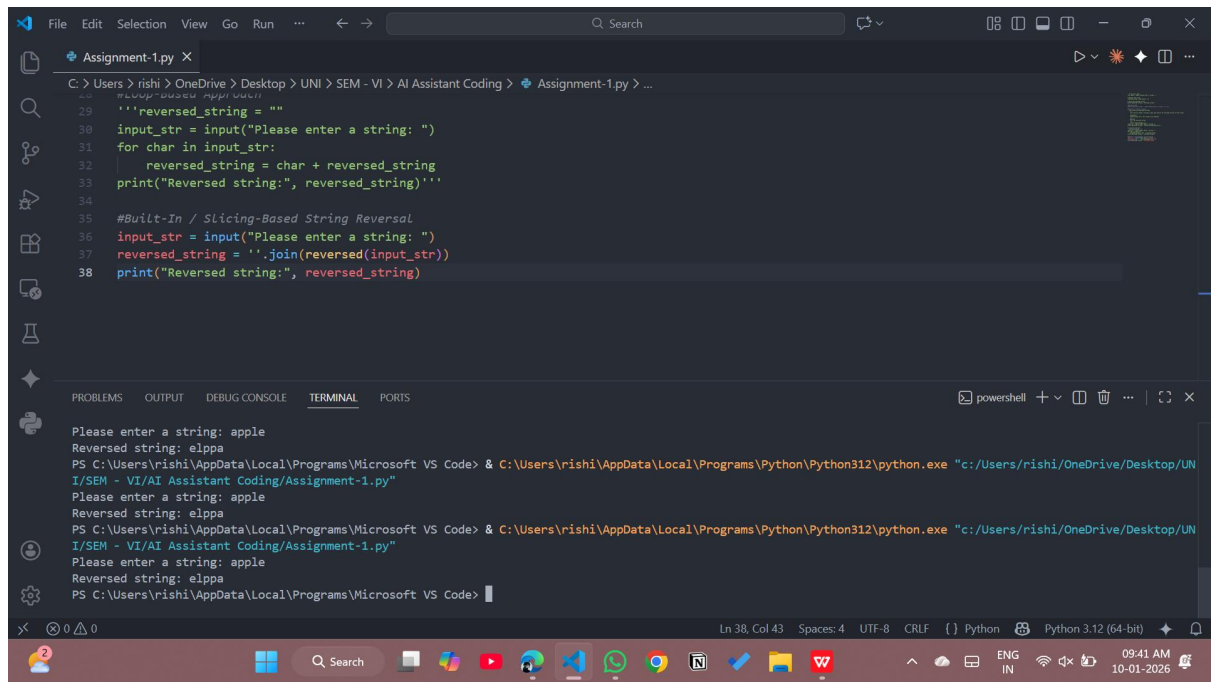
Time Complexity:

- $O(n^2)$ in languages where string concatenation is costly (like Python) because each `char + reversed_string` creates a new string.
- For small strings, this is negligible.

Use Case:

- Good for learning purposes and step-by-step logic demonstration.
- Not ideal for very large strings

Approach 2: Built-In / Slicing-Based String Reversal



```
28 #Loop-based approach
29 reversed_string = ""
30 input_str = input("Please enter a string: ")
31 for char in input_str:
32     reversed_string = char + reversed_string
33 print("Reversed string:", reversed_string)'''
34
35 #Built-In / Slicing-Based String Reversal
36 input_str = input("Please enter a string: ")
37 reversed_string = ''.join(reversed(input_str))
38 print("Reversed string:", reversed_string)
```

Reversed string: elppa

PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment-1.py"

Please enter a string: apple

Reversed string: elppa

PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment-1.py"

Please enter a string: apple

Reversed string: elppa

PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code>

Execution Flow

1. Program reads input from the user.
2. Uses Python slicing `[::-1]` to reverse the string in a single step.
3. Prints the final reversed string.

Time Complexity:

- **$O(n)$** – very efficient even for large strings.

Use Case:

- Best for production code or large-scale applications.
- Cleaner, shorter, and more maintainable than the loop-based approach.

Comparison Table

Criteria	Loop-Based Approach	Slicing-Based Approach
Code Complexity	Longer, step-by-step	Short, single-line
Execution Flow	Iterative prepending of characters	Direct slicing operation
Time Complexity	$O(n^2)$ (due to repeated concatenation)	$O(n)$
Performance (Large Input)	Slower, may cause overhead for large strings	Very fast, scales well
Readability	Moderate	High
Use Case	Learning, step-by-step explanation	Production, large applications

